SDK Implementation Exercise

Partner Implementation Technical Track

Setup:

- Install Sublime
    - This is a lightweight text editor that we'll use to write code.
- Install Amplitude Instrumentation Explorer in Chrome
    - This is a Chrome extension that lets you see your instrumentation live.
- Install Git: https://github.com/
    - Click the link above and create a GitHub account if you don't already have one.
    - Use command + space to open Spotlight, type "Terminal," and press enter. In the terminal window that appears, type `git --version`, which should open an installation prompt.
    - This is a tool commonly used by engineers to manage their code. We'll just use it to get the initial version of the code to work with.
    - If the above doesn't work, follow the instructions here: https://github.com/amplitude/docs/tree/master/topics/eng-team-week-0-setup#github-setup
        - You will have to follow the Homebrew instructions first.
- Download and install Python
- Make a copy of the SDK Assignment Deck Template. You will add screenshots as you go through this exercise.

Run the Battleboat web app

- In your terminal, navigate to an appropriate directory and copy the code for the "Battleboat" web app to get the initial version of the code to work with.

```
git clone https://github.com/billmei/battleboat.git
```

- Then run `cd battleboat/` to go into the directory of the code.

- Start the server using `python3 -m SimpleHTTPServer 8000`
    - If using Python (latest) use `python3 -m http.server`

- Navigate to localhost:8000 in the Chrome browser. You should see the game now! Try playing a game to get a feel for the functionality

- When you change the code, you'll need to save the file and refresh the page to get the updated code. Keep that in mind for the rest of this tutorial!

Install Amplitude with Autocapture

- Open up the `battleboat/` directory in Sublime.

- Open `index.html` and add the Amplitude snippet just before the </head> tag near or on line 23. 5511adbb84b54c7cdf75ccd44ea02141
    - Go to your test Amplitude instance and create a project. Name it "Battleboat."
    - Replace the API key with that project's API Key.
        - Instructions on creating a project are available here. API keys can be found by navigating to ***Organization Settings -> Projects -> Battleboat*** in your Amplitude instance.

- Just by adding the Amplitude Snippetfinder
- , you will be able to use Autocapture. When you refresh or relaunch the app, you will see an [Amplitude] Page Viewed event in the Chrome extension.



Track an Event:

- Next, open `js/battleboat.js`. We will be working on this file for the rest of the tutorial.

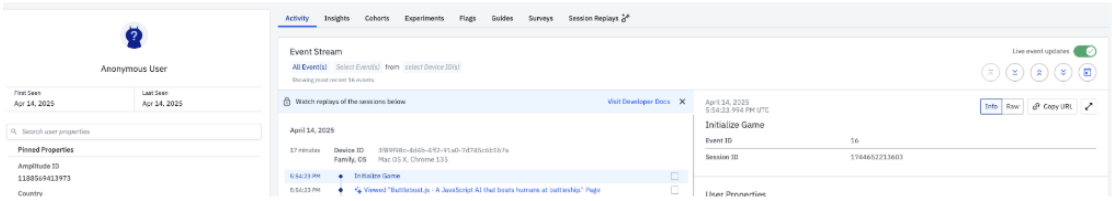
- Now, it's time to instrument our first event!
  On line 173 (approximately), line in `function Game(size)`, instrument the `Initialize Game` event using a `track` call. Make sure it is after `this.init()`:

```
amplitude.track('Initialize Game');
```

- This is called after the game is initialized when the page loads. Now, every time you refresh the page, it will fire this event. Check the Chrome extension.



- You may also monitor the event stream in your Amplitude instance. Extract the device ID from your Chrome extension installation and enter it in the User Lookup search field. This will display a real-time stream of all user interaction events associated with that device.



- Congratulations, you have now “instrumented” Amplitude!

Instrument Ship Placement events with event properties:

- Now, let’s add more events to our instrumentation to make it more useful.
- Let’s start with the `Select Ship` event. On line 276 (within the `Game.prototype.rosterListener = function(e)` function), add this:

```
// Track ship selection event
const eventProperties = {
    ship: Game.placeShipType
};
amplitude.track('Select Ship', eventProperties);
```

- We’ve instrumented our first event property! Verify this by selecting the ship to place and seeing your events show up in the Chrome Extension
- Let’s add a few more events now.
- Instrument the `Rotate Ship` event.
  Add the following code to track the event on line 382 (in the `toggleRotation` function),

```
// Track ship rotation event
const eventProperties = {
      ship: Game.placeShipType,
      direction: Game.placeShipDirection ===
Ship.DIRECTION_VERTICAL ? 'vertical':'horizontal'};
amplitude.track('Rotate Ship', eventProperties);
```

Save the file, refresh the web app, and validate this by clicking on the “Rotate Ship” button

- Instrument the `Place Ship` event.
  Add the following code first to ensure (in the `placementListener` function) that variables exist to be used as event properties:

```
// Store the ship type before it gets reset
var shipType = Game.placeShipType;
var shipDirection = Game.placeShipDirection ===
Ship.DIRECTION_VERTICAL ? 'vertical' : 'horizontal';
```

- To track the event, add following code on line 305 (in the `placementListener` function):

```
// Track ship placement event
const eventProperties = {
      ship: shipType,
      success: true,
      direction: shipDirection };
amplitude.track('Place Ship', eventProperties);
```

Save the file, refresh the web app, and validate this by clicking on the “Rotate Ship” button

- Instrument the `Place Ship` failure event
  Add the following code at line 331 to track the event with an `else` condition. Ensure that else is added after the `if` block closes (after `}`)

```
else {
// Track failed ship placement event
const failureEventProperties = {
      ship: shipType,
      success: false,
      direction: shipDirection };
amplitude.track('Place Ship', failureEventProperties);
}
```

- Great, now the whole ship placement flow is instrumented!

Instrument gameplay events

- Now, let's instrument the rest of the game to be complete.

- Instrument the `Start Game` event:
  Add the following code at line 417 to track the event in `startGame` function

```
// Track game start event
const eventProperties = {
      allShipsPlaced: true };
amplitude.track('Start Game', eventProperties);
```

- Instrument the `Shoot Ship` Event:
  Add the following code at line 246 to track the event in `shootListener` function (in the `if` condition). The properties with this event (x,y) also track the location of the shot.

```
// Track shot event
const eventProperties = {
      x: x,
      y: y,
      hit: result === CONST.TYPE_HIT,
      miss: result === CONST.TYPE_MISS
};
amplitude.track('Shoot Ship', eventProperties);
```

- Instrument the `Sink Opponent Ship` Event:
  Add the following code at line 885 to track the event in `sinkShip` function

```
// Track when player sinks a computer ship
if (this.player === CONST.COMPUTER_PLAYER) {
      const eventProperties = {
            ship: this.type,
            shipLength: this.shipLength};
      amplitude.track('Sink Opponent Ship', eventProperties);
      }
```

- Instrument `Game End` Event

Add the following code at line 184 to track the event in `checkIfWon` function to add an event if you win the game:

```
// Track game end event - player won
const eventProperties = {
      result: 'win',
      shotsTaken: Game.stats.shotsTaken
      };
amplitude.track('Game End', eventProperties);
```

In the same function, add the following code at line 199 to track the event if you lose the game:

```
// Track game end event - player lost
const eventProperties = {
      result: 'win',
      shotsTaken: Game.stats.shotsTaken
      };
amplitude.track('Game End', eventProperties);
```

Add User Property:

- Finally, let's add some user properties to your instrumentation.
- We will add three user properties: `Total Games Played`, `Total Games Won`, and `Total Games Lost`
- These are all incremental user properties which means they will increment with each completed game.
- Remember, the timing when you set these properties is crucial as it will decide when these user properties are available with events. We want the user properties to be available with `Game End` event so we trigger them right before logging the events
- To track the properties associated with winning a game add the following code in `checkIfWon` function on line 185

```
// Create identify object for user properties
var identify = new amplitude.Identify();
```

```
identify.add('Total Games Played', 1);
identify.add('Total Games Won', 1);
amplitude.identify(identify);
```

- To track the properties associated with losing a game add the following code in `checkIfWon` function on line 206

```
// Create identify object for user properties
var identify = new amplitude.Identify();
identify.add('Total Games Played', 1);
identify.add('Total Games Lost', 1);
amplitude.identify(identify);
```

You have finished instrumenting Amplitude. Play the game completely to validate all events and properties are working as expected.